

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Large Language Models (LLMs) are rapidly becoming core components for graph-structured tasks such as node classification, a setting historically dominated by Graph Neural Networks (GNNs). This integration strengthens graph reasoning by combining relational structure with language-based representations, but it also introduces a robustness problem that has not been systematically examined. We provide the first comprehensive study of adversarial vulnerabilities in graph-aware LLMs by adapting established graph adversarial attacks for both poisoning (training-time) and evasion (test-time) settings to two representative models, LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024). We further identify a distinct attack surface in LLAGA, where malicious nodes can be injected as placeholders in the node sequence template and thereby sharply degrade performance. Our analysis establishes that graph encoding choices directly shape attack success: (1) the node sequence template in LLAGA creates pronounced vulnerability; (2) the GNN encoder in GRAPHPROMPTER provides stronger structural robustness; and (3) both paradigms remain highly exposed to imperceptible feature perturbations. Finally, we introduce GALGUARD, an end-to-end defense framework that couples an LLM-based feature correction module with adapted GNN defenses to mitigate both feature-level and structural attacks¹

1 Introduction

Graphs use nodes and edges to capture entities and relationships, making them a foundational representation for complex systems in social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and healthcare (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018) have therefore become the standard machinery for graph tasks such as node classification and link prediction. At the same time, the success of large language models (LLMs) such as GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023), and PaLM-2 (Anil et al. 2023) has created a decisive shift toward using language models for graph learning. This fusion of graph data with LLMs, referred to as *graph-aware LLMs* (Xie et al. 2023), defines an emerging

model class in which LLMs perform tasks previously handled primarily by GNNs (Ye et al. 2023; Fatemi, Halcrow, and Peruzzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). Recent results show that graph-aware LLMs can outperform GNNs on several graph-related tasks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), positioning them as credible and powerful alternatives.

Despite this momentum, the adversarial robustness of graph-aware LLMs remains largely uncharted. Existing graph adversarial attacks, including poisoning attacks during training and evasion attacks at test time (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023), commonly operate in a black-box setting where the attacker lacks access to internal model details. These attacks compromise *model integrity* by degrading performance through subtle changes to graph structure or node features. While the GNN literature has emphasized structural perturbations, graph-aware LLMs inherit an additional and consequential exposure: their textual node features can be attacked through imperceptible perturbations that are largely invisible to humans but disruptive to language-model processing.

These observations motivate our central question: *do adversarial attacks designed for GNNs remain effective against the emerging graph-aware LLM paradigm?* Answering this question requires attention to how graph information is encoded for the LLM. Node features can be consumed naturally by language models, but graph structure must be injected through specialized mechanisms that preserve relational information (Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). We therefore categorize graph-aware LLMs into two encoding families: methods that express graphs as *textual descriptions*, and methods that use *learned projectors* to bridge graph and language representations. Because conventional text attacks already apply to textual graph descriptions, we focus on learned projector methods, where graph-specific design decisions create distinctive robustness questions. Our study uses two computationally efficient representative systems, LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024), both of which rely on frozen pre-trained LLMs.

Our analysis shows that graph encoding design is not merely an implementation detail; it is a primary determinant

¹Our code will be released upon acceptance.

of adversarial vulnerability. We identify a new attack surface in LLAGA that enables an attacker to inject malicious nodes as placeholders in the node sequence template and substantially degrade performance. We also introduce an end-to-end graph-aware LLM defense, GALGUARD, which combines an LLM-based feature corrector with adapted GNN-based structural defenses. To our knowledge, this is the first systematic investigation of adversarial attacks and defenses for graph-aware LLMs. The contributions of the paper are summarized as follows:

- We adapt representative GNN adversarial attacks, specifically NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to evaluate both poisoning and evasion attacks on graph-aware LLMs.
- We establish a graph encoding-based taxonomy of graph-aware LLMs that clarifies how different methods integrate structural information into LLMs.
- We identify and evaluate a new attack surface in LLAGA, demonstrating that malicious placeholder injection in the node sequence template can markedly degrade performance.
- We deliver a holistic robustness assessment by studying imperceptible perturbations to textual node features, a vulnerability channel that is typically secondary in GNN-focused analyses.
- We propose GALGUARD, a new end-to-end defense strategy that integrates LLM-based feature correction with adapted GNN-based structural defenses.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

Graph encoding methods for LLMs fall into two principal families: *textual descriptions of graphs*, which encode graph information as natural language, and *learned graph projectors*, which map graph data into embeddings or structured representations that LLMs can process directly. This taxonomy provides the basis for isolating the robustness mechanisms that matter for graph-aware LLMs. Figure 5 illustrates both families, with a fuller description in Appendix A.1 and related work on GNN attacks in Appendix A.2.

Graph Representation as a Textual Description. Textual-description methods convert graph data into natural language so that LLMs can process the graph through prompts and instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). In multimodal settings, Liu et al. (2023) and Su et al. (2022) align molecule graphs with textual descriptions. Huang et al. (2023a) further study how graph structure can be injected into LLM prompts, showing that LLMs often process graph prompts as contextual paragraphs and therefore depend strongly on prompt design, as also reflected in (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). Other work uses LLMs to enrich node features, refine graph structures, serve as predictors, or extract features

for GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). For inference, Zhu et al. (2024) propose a fine-tuning method for textual graphs that combines LLMs and GNNs, while Duan et al. (2023) introduce an effective approach for textual graph learning. These methods provide a direct path for applying LLMs to graph tasks, but they often struggle to represent structural complexity at scale and remain heavily dependent on prompt engineering.

Graph Representation as a Learned Projector. Learned-projector methods address the limitations of textual descriptions by transforming graph data into embeddings or graph vectors that can be supplied to the LLM more directly. Representative examples include LLAGA, which uses node-level templates with a linear projector (Chen et al. 2024), and methods that insert GNN layers into a pre-trained LLM pipeline (Liu et al. 2024; Qin et al. 2023). Additional approaches use graph neural prompts for knowledge graphs (Tian et al. 2024), prompt-based node feature extractors with a GNN adapter (Huang et al. 2023b), or graph transformers that condense graph information into fixed-length prefixes (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024). This family is central to our study because the projector and graph encoder determine how structural perturbations reach the LLM.

This Work: We focus on projector methods with *frozen pre-trained LLMs* because they are computationally efficient, avoid expensive retraining, and represent a practical deployment path (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). We study two representative models: LLAGA (Chen et al. 2024), which uses node-level templates with a linear projector, and GRAPH-PROMPTER (Liu et al. 2024), which uses a GNN with a linear projector. We exclude textual-description methods because their scalability limitations and conventional text-attack exposure make them less informative for the graph-specific vulnerabilities targeted in this work.

3 Threat Model

We define a threat model grounded in established graph adversarial attack research and extend it directly to graph-aware LLMs (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). In social networks, for example, an attacker may control only a small number of bot accounts; this limited control matches our setting, where the adversary manipulates a small subset of nodes to evade detection or influence the classification of legitimate users. In Wikipedia-style graphs, sparse hoax articles can be strategically linked to genuine articles to alter the predicted category of real content. These scenarios matter because graph-aware LLMs are plausible replacements for GNNs in high-impact domains such as recommendation, social media analysis, and healthcare. Following the graph adversarial literature, we specify the threat model along model access, adversarial capabilities, attack strategies, and attacker goals.

Model Access. The attacker operates in a black-box setting with no knowledge of the model architecture or parameters.

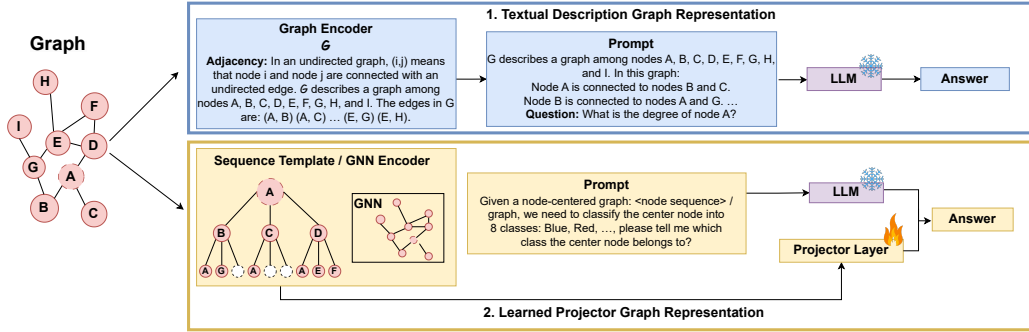


Figure 1: **Graph encoding-based adaptations of LLMs.** Textual-description methods express graph information in natural language (top). Learned-projector methods encode graphs through node templates that form sequences or through GNN encoders (bottom). In both families shown here, the LLM remains frozen and only the projector that maps graph information into tokens is trained.

The attacker can affect the model only indirectly through the training or test data.

Adversarial Capabilities. Consistent with existing poisoning and evasion attacks, the attacker has access to the training data in the poisoning setting and to the inference data in the evasion setting. Within a predefined budget and under unnoticeability constraints, the attacker can modify or inject data.

Attack Strategies. The attacker uses two primary strategies: feature manipulation, which alters node features, and structure manipulation, which adds or removes edges.

Attacker’s Goal. The attacker aims to compromise model integrity by degrading overall performance, specifically by increasing the misclassification rate of the classification model.

4 Experimental Setup

Dataset. We evaluate on three widely used graph benchmarks: Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016). These text-attributed graphs span small to medium scales with different sparsity levels, making them appropriate and established testbeds for the graph-aware LLMs studied here (Chen et al. 2023a, 2024; Liu et al. 2024). To test whether the observed attack behavior extends beyond these benchmarks, we additionally evaluate on the larger ArXiv dataset (Hu et al. 2020) in Appendix A.5. We use the standard train/validation/test splits of 6:2:2 for the three smaller datasets and 5:2:3 for ArXiv, following (Chen et al. 2024). Table 1 reports the dataset statistics.

Table 1: Statistics of the graph datasets used for the main evaluation.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

Tasks. We use *node classification* as the primary task because it is a standard evaluation setting for graph machine learning and the canonical setting for the adversarial

attacks considered in this study (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023).

Evaluation Metrics. We report *accuracy*, the proportion of correctly predicted labels, as the evaluation metric. Each experiment is repeated three times, and we report the mean and standard deviation to make the empirical trends explicit. All experiments were conducted on a machine equipped with an NVIDIA A40 GPU with 48GB of memory.

Choice of Graph-aware LLM and Adversarial Attack Methods. We deliberately pair representative graph-aware LLM architectures with representative graph adversarial attacks so that the evaluation isolates how graph encoding choices shape vulnerability.

Graph-Aware LLM Methods. We study two representative graph-aware LLM methods: LLAGA (Chen et al. 2024) and GRAPH PROMPTER (Liu et al. 2024). Both use a *frozen pre-trained LLM*, which makes them computationally efficient and adaptable across graph tasks and LLM backbones. LLAGA encodes local structure through node-level templates and a linear projector, whereas GRAPH PROMPTER uses a GNN-based encoder followed by a linear projector. In both models, the projector is a 2-layer multi-layer perceptron (MLP). The foundation LLMs for LLAGA and GRAPH PROMPTER are Vicuna-7B (Chiang et al. 2023) and LLAMA2-7B (Touvron et al. 2023), respectively.

Adversarial Attack Methods. We use poisoning and evasion variants of two representative adversarial attack methods, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to probe graph-aware LLM robustness. NETTACK is a targeted attack that identifies minimal perturbations for misclassifying selected nodes, while METAATTACK performs untargeted attacks by optimizing graph structure as a hyperparameter. For NETTACK, we follow (Li et al. 2023; Zügner and Günnemann 2019) by sequentially targeting each node and aggregating the results, thereby converting it into an untargeted evaluation. Across experiments, we perturb only 10% of the edges to preserve the unnoticeability constraint used in (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann

Table 2: Accuracy of graph-aware LLMs LLAGA and GRAPH PROMPTER under NETTACK and METAATTACK on Cora, Citeseer, and PubMed.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

2019). Appendix A.2 provides the full attack details.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

We evaluate poisoning and evasion attacks by adapting NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019) to the graph-aware LLM setting. For poisoning attacks on LLAGA, the perturbed graph is used to generate node-level sequences before retraining the projector. For GRAPH PROMPTER, the perturbed graph is re-encoded with a GAT and the projector is retrained. For evasion attacks, both models are trained on clean data and encounter perturbations only at inference time, directly testing deployment-time fragility.

Results. Table 2 yields three clear findings. First, poisoning attacks have limited impact on these graph-aware LLMs, with maximum degradation of 11% across the evaluated attack methods, indicating that retraining the projector with perturbed data retains some resilience from the learned representations and LLM context. Evasion attacks are substantially more damaging, producing performance drops as large as 51%. Second, METAATTACK generally outperforms NETTACK for both poisoning and evasion across datasets, consistent with its ability to produce small but high-impact perturbations by treating graph edges as hyperparameters. Third, denser datasets are more exposed than sparser ones: under evasion, LLAGA drops by 51% on denser Cora but by only 19% on sparser PubMed.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

Our experiments (Table 2) identify LLAGA as the more structurally vulnerable paradigm, primarily because its representation is tightly coupled to the perturbed graph. LLAGA injects structure by generating a fixed-length node-level sequence for each node and by computing a graph Laplacian embedding; each sequence position corresponds to a specific relative structural role in the original graph. When the graph is perturbed, the resulting node embeddings directly alter the information passed to the LLM, producing the observed degradation. GRAPH PROMPTER is more robust because its textual node embeddings remain unchanged under structural

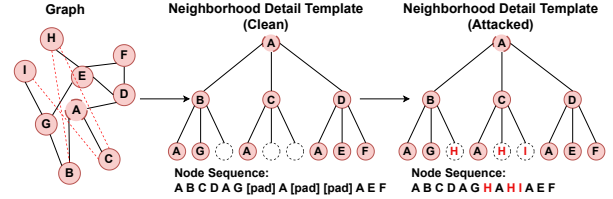


Figure 2: Node sequence template injection attacks. The example shows Non-adjacent Injection (NI), where red dashed lines denote attacker-added edge perturbations. In Supernode Injection (SI) and Multiple Supernode Injection (MSI), one or more supernodes replace template placeholders. Here, $k = 3$.

perturbations and only the GNN component is perturbed, allowing clean textual features to compensate for corrupted structural signals.

6 Node Sequence Template Injection Attacks

We expose a new attack surface in LLAGA by targeting the neighborhood detail template that constructs node sequences. Figure 2 illustrates the attack, and Appendix A.4 provides the algorithm. To obtain the node sequence for a node u , LLAGA first builds a *fixed-shape* computational tree whose children are sampled 1-hop neighbors; the same sampling process repeats at subsequent tree levels. When a node lacks enough neighbors, LLAGA inserts placeholders to preserve a fixed size, such as $k = 10$ children, following the original design (see "Clean" in Figure 2 and lines 1-14 in the algorithm). This placeholder mechanism creates a concrete and exploitable structural weakness.

This attack manipulates the graph so that malicious nodes occupy placeholder positions in LLAGA’s node sequence, sharply disrupting the target representation. We instantiate the attack with three strategies. *Non-Adjacent Injection (NI)* injects nodes outside the 2-hop neighborhood of a target node u (see "Attacked" in Figure 2 and lines 21-27). *Supernode Injection (SI)* injects a high-degree node as a placeholder while keeping it outside the target node’s 2-hop neighborhood. *Multiple Supernode Injection (MSI)* extends SI by injecting multiple distinct supernodes as placeholders (lines 35-43). The mechanism is direct: uninformative injected nodes corrupt the node sequence, degrade the target-node representation,

Table 3: Accuracy under node sequence template injection attacks across datasets. *NI*, *SI*, and *MSI* denote non-adjacent, single-supernode, and multiple-supernode injection attacks, respectively. Clean reports unattacked model performance.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 $\pm$ 0.07	—
	Poisoning	0.86 $\pm$ 0.03	0.84 $\pm$ 0.06	0.82 $\pm$ 0.04
	Evasion	0.42 $\pm$ 0.08	0.36 $\pm$ 0.05	0.30 $\pm$ 0.05
Citeseer	Clean	—	0.71 $\pm$ 0.04	—
	Poisoning	0.69 $\pm$ 0.05	0.66 $\pm$ 0.03	0.62 $\pm$ 0.02
	Evasion	0.36 $\pm$ 0.06	0.30 $\pm$ 0.04	0.24 $\pm$ 0.05
PubMed	Clean	—	0.90 $\pm$ 0.04	—
	Poisoning	0.88 $\pm$ 0.04	0.85 $\pm$ 0.05	0.81 $\pm$ 0.03
	Evasion	0.85 $\pm$ 0.07	0.82 $\pm$ 0.04	0.76 $\pm$ 0.03

and weaken the projection layer used for both learning and inference.

Results. As shown in Table 3, LLAGA remains relatively resilient to poisoning attacks on Cora and Citeseer, but it is highly vulnerable under evasion. Accuracy falls to 0.42, 0.36, and 0.30 on Cora and to 0.36, 0.30, and 0.24 on Citeseer for *NI*, *SI*, and *MSI*, respectively.

On the larger PubMed dataset, LLAGA exhibits stronger resilience to all injection attacks, with evasion accuracies of 0.85, 0.82, and 0.76. This contrast indicates that localized injection is especially damaging in smaller, more tightly connected graphs such as Cora and Citeseer, where the perturbation has a concentrated effect on local neighborhood reasoning. In the much larger PubMed graph, the same perturbation is more diluted and therefore produces a smaller degradation.

Node injection attacks are more damaging than the traditional graph adversarial methods in several settings. For example, *MSI* yields evasion accuracies of 0.30, 0.24, and 0.76 on Cora, CiteSeer, and PubMed, respectively, compared with 0.44, 0.50, and 0.73 under METAATTACK (Table 2).

7 Poisoning and Evasion Attack via Feature Perturbation

Existing graph adversarial attacks mainly target structure (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019); NETTACK (Zügner, Akbarnejad, and Günnemann 2018) is a notable exception, but it does not transfer directly to textual features. Because structural attacks are only moderately effective against more robust models such as GRAPHPROMPTER, and because textual features clearly contribute to robustness, we directly test imperceptible feature perturbations as a major attack channel for graph-aware LLMs.

We adopt the imperceptible attack strategies of (Boucher et al. 2022) with a key modification: perturbations are generated using a task-agnostic English-to-French translation objective. The attack modifies input text so that it remains visually identical to a human reader but causes an incorrect translation, thereby corrupting the encoded semantics that

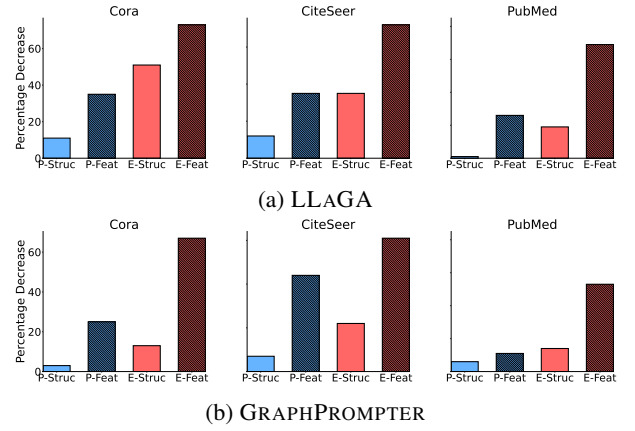


Figure 3: Structural and feature-based adversarial attacks on graph-aware LLMs. Attack success rate is measured as the percentage decrease from clean accuracy on the y-axis. P-Struc, P-Feat, E-Struc, and E-Feat denote poisoning and evasion attacks based on structural and feature perturbations.

the LLM receives. This objective captures the core risk for graph-aware LLMs: if the language model mis-encodes the node text, the downstream graph prediction can fail. We evaluate two imperceptible attacks: *Homoglyphs*, which replace characters with visually similar alternatives, and *Reorderings*, which use direction-control characters to alter rendering order. Appendix A.6 provides examples of Homoglyphs and Reorderings.

Following (Boucher et al. 2022), we optimize perturbations with differential evolution (Storn and Price 1997), maximizing the Levenshtein distance between model outputs on perturbed and unperturbed text. We set the perturbation budget to 10% of the average length of each textual feature, preserving the intended imperceptibility constraint.

Results. Table 4 shows that Reordering attacks are consistently stronger than Homoglyph attacks across models and datasets. On Cora and CiteSeer, LLAGA is vulnerable to both attacks: Homoglyph poisoning reduces accuracy by 27% and 23%, while Reordering poisoning reduces it by 35% and 25%, respectively. GRAPHPROMPTER is more resilient in these cases, but it is still clearly affected. PubMed follows the same qualitative pattern, with a smaller overall impact than the smaller datasets.

The comparison with structural perturbations in Figure 3 reveals a decisive shift in the vulnerability profile of graph-aware LLMs. Feature perturbations, especially Reordering, outperform structural METAATTACK perturbations by a large margin on both LLAGA and GRAPHPROMPTER. This evidence shows that graph-aware LLMs depend heavily on textual node features and are therefore more exposed to feature-level attacks than traditional GNN-centric threat models would suggest. The next section tests this conclusion in a unified attack that combines feature and structural perturbations.

Table 4: Accuracy under imperceptible feature perturbation attacks, with percentage drop from clean accuracy shown in parentheses.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 ± 0.05 (27%)	0.52 ± 0.04 (13%)	0.41 ± 0.04 (54%)	0.38 ± 0.03 (37%)
	Reordering	0.58 ± 0.06 (35%)	0.45 ± 0.05 (25%)	0.24 ± 0.05 (73%)	0.20 ± 0.03 (67%)
Citeseer	Homoglyph	0.54 ± 0.04 (23%)	0.65 ± 0.04 (7%)	0.32 ± 0.03 (55%)	0.52 ± 0.05 (26%)
	Reordering	0.46 ± 0.03 (35%)	0.39 ± 0.02 (44%)	0.20 ± 0.05 (72%)	0.27 ± 0.02 (61%)
PubMed	Homoglyph	0.72 ± 0.04 (20%)	0.83 ± 0.04 (8%)	0.36 ± 0.06 (60%)	0.55 ± 0.03 (39%)
	Reordering	0.67 ± 0.05 (26%)	0.80 ± 0.03 (11%)	0.28 ± 0.04 (69%)	0.42 ± 0.04 (53%)

Table 5: Model accuracy under the unified imperceptible attack, with percentage drop from clean accuracy shown in parentheses.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 ± 0.05 (61%)	0.28 ± 0.08 (53%)
	Evasion	0.14 ± 0.06 (84%)	0.11 ± 0.05 (82%)
Citeseer	Poisoning	0.26 ± 0.04 (63%)	0.22 ± 0.03 (69%)
	Evasion	0.10 ± 0.09 (86%)	0.13 ± 0.07 (81%)
PubMed	Poisoning	0.39 ± 0.07 (57%)	0.32 ± 0.04 (64%)
	Evasion	0.12 ± 0.05 (87%)	0.19 ± 0.04 (79%)

8 A Unified Imperceptible Attack on Graph-aware LLMs

We introduce a unified attack that combines feature and structural perturbations, directly targeting the two information channels used by graph-aware LLMs. The goal is to determine whether jointly perturbing text and structure produces stronger degradation than either attack family alone while preserving imperceptible changes to both graph structure and node features.

For structural perturbation, we use METAATTACK, which subtly adjusts graph edges. For feature perturbation, we combine Homoglyph and Reordering attacks as described in the previous section. The resulting attack simultaneously corrupts relational and textual signals, matching the multimodal dependence of graph-aware LLMs.

Results. Table 5 demonstrates that jointly applying structural and feature perturbations is substantially more damaging than either perturbation family alone across datasets. The result clarifies a central architectural property of graph-aware LLMs: unlike traditional GNNs, whose robustness is often dominated by structural perturbations, these models are strongly exposed to feature-level manipulation. The unified attack succeeds because graph-aware LLMs rely on the consistency of both graph structure and node text, making coordinated inconspicuous perturbations especially effective.

9 Ablation Study

9.1 Varying Feature Perturbation

To characterize how attack strength scales with perturbation budget, we vary the percentage of perturbed features. The Reordering attack, which follows the same qualitative pattern as Homoglyph, shows a consistent trend as the perturbation percentage ranges from 1% to 10%. Across LLAGA, GRAPHPROMPTER, and all three datasets (Cora, Citeseer, and PubMed), reducing the perturbed-feature percentage weakens the attack. As the budget decreases, poisoning and evasion success rates also converge, indicating that small perturbation budgets produce an adversarial signal too weak to dominate either training-time or inference-time behavior. The Appendix (Figure 2) provides the detailed plot.

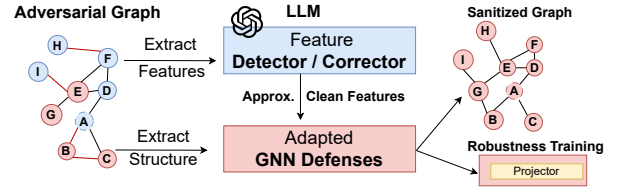


Figure 4: **Overview of GALGUARD defense.** Perturbed node features and edges are shown in blue and red, respectively. The LLM detects anomalous node-feature text from an adversarial graph and produces corrected features. The corrected features and perturbed edges are then processed by adapted GNN defenses, which either sanitize the graph or support robust projector training.

9.2 Does the Choice of LLM Affect Attack Performance?

We evaluate whether the foundation LLM meaningfully changes adversarial robustness in graph-aware LLMs. LLAGA uses Vicuna-7B (Chiang et al. 2023), while GRAPHPROMPTER uses Llama2-7B (Touvron et al. 2023). We replace LLAGA’s backbone with Llama2-7B and compare clean, poisoning, and evasion performance on all datasets.

The backbone swap produces nearly identical clean accuracy (e.g., Vicuna: 0.89 ± 0.07 , Llama2: 0.90 ± 0.03 on Cora) and comparable degradation under attack. The same trend holds when swapping LLMs in GRAPHPROMPTER. These results indicate that the graph encoding mechanism, rather than the specific foundation LLM among those tested, is the dominant driver of adversarial robustness. Table 2 in the Appendix reports the full results.

10 Defense: GALGUARD

Existing GNN defenses are insufficient for graph-aware LLMs because these models are jointly vulnerable to structural and feature perturbations. Many defenses prune or reweight edges using node-feature similarity (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), which implicitly assumes access to clean features. Once textual features are perturbed, these defenses lose the signal they rely on (see Figure 3 in the Appendix). Defenses that decouple structure and features (Shanthamallu,

Thiagarajan, and Spanias 2021; Wu et al. 2021), as well as OCR-based sanitization (Boucher et al. 2022), similarly fail under coordinated attacks. This dual exposure makes an integrated defense necessary rather than optional.

We therefore propose graph-aware LLM defense (GALGUARD), an end-to-end defense that combines two complementary components: (1) an LLM-based feature corrector and (2) adapted GNN-based structural defenses.

Feature Robustness Strategy. As shown in Figure 4, the LLM component acts as an adaptive corrector for anomalous textual node features. It produces approximate corrections that restore feature integrity and reduce the effect of imperceptible feature perturbations. This feature-correction step directly addresses the attack channel that standard GNN defenses do not cover. Appendix A.6 provides the instructions and prompts. In our experiments, we use GPT-4 Turbo as the LLM corrector.

Graph Structure Robustness Strategy. To strengthen structural robustness, GALGUARD uses a two-step strategy. First, it applies a *graph purification heuristic* (Wu et al. 2019) as preprocessing: for each node v , we compute the cosine similarity between its embedding $\phi(x_v)$ and each neighbor embedding $\phi(x_u)$, then remove low-similarity edges. Second, it integrates *GNNGuard* (Zhang and Zitnik 2020). For GRAPH-PROMPTER, we apply GNNGuard’s principles directly to the message-passing layers. For LLAGA, we introduce a model-specific adaptation by incorporating purification into the computational tree used for node sequence construction. We also add a learnable *global structural context embedding* (M_{global}) as a parameter in LLAGA’s projector. Concatenating M_{global} with the purified node sequence embedding and training them jointly enables the model to operate more robustly on potentially perturbed graph inputs. The plug-and-play design of GALGUARD also allows other defenses, such as Pro-GNN (Jin et al. 2020), to replace GNNGuard.

10.1 Results

Table 6 reports model accuracy under METAATTACK alongside two defense variants: GALGUARD_p, which combines the feature corrector with graph purification, and full GALGUARD, which adds GNNGuard or its model-specific adaptation. Across Cora, CiteSeer, and PubMed, and across both poisoning and evasion, the defenses consistently reduce the impact of METAATTACK. GALGUARD_p already provides a strong baseline improvement, confirming that feature correction plus initial graph purification addresses a major portion of the vulnerability. Full GALGUARD delivers the strongest robustness gains, demonstrating the value of coupling the LLM feature corrector with a GNN-inspired structural defense.

11 Conclusion

This paper delivers a systematic first study of adversarial attacks and defenses for graph-aware LLMs. The results establish that graph integration mechanisms are a central source of vulnerability: the node sequence template in LLAGA creates a concrete attack surface, structural perturbations affect graph-aware LLMs unevenly across encoding paradigms, and

Table 6: Accuracy under METAATTACK and the proposed GALGUARD variants for poisoning and evasion attacks. GALGUARD_p combines feature correction with graph purification.

Data	Method	LLAGA		GRAPH-PROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79 $\pm$ 0.03	0.44 $\pm$ 0.06	0.58 $\pm$ 0.05	0.52 $\pm$ 0.04
	GALGUARD _p	0.82 $\pm$ 0.05	0.62 $\pm$ 0.04	0.59 $\pm$ 0.03	0.56 $\pm$ 0.04
	GALGUARD	0.85$\pm$0.04	0.83$\pm$0.04	0.60$\pm$0.05	0.59$\pm$0.03
CiteSeer	MetaAttack	0.63 $\pm$ 0.04	0.50 $\pm$ 0.03	0.65 $\pm$ 0.06	0.56 $\pm$ 0.03
	GALGUARD _p	0.64 $\pm$ 0.03	0.56 $\pm$ 0.02	0.67 $\pm$ 0.05	0.60 $\pm$ 0.06
	GALGUARD	0.67$\pm$0.04	0.62$\pm$0.04	0.68$\pm$0.04	0.64$\pm$0.05
PubMed	MetaAttack	0.89 $\pm$ 0.08	0.73 $\pm$ 0.04	0.85 $\pm$ 0.02	0.77 $\pm$ 0.03
	GALGUARD _p	0.90 $\pm$ 0.06	0.78 $\pm$ 0.03	0.85 $\pm$ 0.04	0.81 $\pm$ 0.04
	GALGUARD	0.90$\pm$0.01	0.87$\pm$0.02	0.89$\pm$0.02	0.89$\pm$0.01

imperceptible textual feature perturbations are often more damaging than traditional structural attacks. The unified attack further shows that coordinated feature and structure perturbations can severely compromise graph-aware LLM performance.

We address these vulnerabilities with GALGUARD, an end-to-end defense that integrates an LLM-based feature corrector with adapted GNN-based structural defenses. As graph-aware LLMs move toward applications in social networks, healthcare, finance, and other high-impact domains, adversarial resilience must be treated as a core design requirement. Our findings establish a foundation for future work on robust graph-aware LLMs and provide concrete evidence that both graph encoding and textual feature integrity must be defended for reliable deployment.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.
- Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.
- Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.
- Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised con-

- trastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.
- Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.
- Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.
- Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.
- Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.
- Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.
- Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.
- Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.
- Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.
- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.
- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.

- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure-text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejdli, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.
- Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.
- Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.
- Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.
- Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.

- Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.
- Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.
- Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.
- Zhang, X.; and Zitnik, M. 2020. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.
- Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.
- Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.
- Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.
- Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.
- Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.
- Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.
- Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix provides the supporting detail for the paper’s attack and defense claims. Appendix A.1 gives the full taxonomy of graph-aware LLMs, and Appendix A.2 reviews attacks against Graph Neural Networks (GNNs). Appendix A.3 details NETTACK and METAATTACK, while Appendix A.4 specifies the node sequence injection attack on LLAGA. Appendix A.5 reports ablations on feature-perturbation budgets and base LLM choice. Finally, Appendix A.7 presents feature-attack examples and the LLM corrector template, and Appendix A.8 discusses the broader implications of the results.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The integration of large language models (LLMs) with graph data has matured into a distinct research direction for graph reasoning, representation, and inference. Existing methods can be cleanly divided by how they encode graphs for LLM consumption: *textual descriptions of graphs*, which express graph information as natural language, and *learned graph projectors*, which map graph data into embeddings or structured tokens that LLMs can process directly. This taxonomy is central to our robustness analysis because the encoding pathway determines which adversarial perturbations can reach the model. Figure 5 illustrates both approaches, and Table 7 summarizes their main differences.

Graph Representation as a Textual Description In this category, graph data is translated into natural language, allowing LLMs to process graph structures through textual prompts. This approach has been widely explored in work that converts graph data into text and pairs it with task-specific instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a). For example, Tan et al. (2023) introduced a unified attributed-graph embedding framework that generates random-walk textual sequences and fine-tunes LLMs on them. Zhao et al. (2023) similarly translated graph structures into natural language for graph reasoning in text space. Wei et al. (2024a) extended textual graph representations with visual graph renderings, improving LLM reasoning on graph data. In multimodal applications, Liu et al. (2023) and Su et al. (2022) linked molecule graphs with textual descriptions for drug discovery and molecular retrieval. Huang et al. (2023a) further showed that graph-structured prompts are often processed by LLMs as contextual paragraphs rather than explicit graph objects, underscoring the decisive role of prompt design in heterogeneous and large-scale graph settings (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). Complementing these efforts, (He et al. 2024) used LLM-generated explanations as features to improve both performance and interpretability on graph tasks.

A related line of textual-description work combines LLMs with GNN pretraining strategies to strengthen graph reasoning. Chen et al. (2023a) proposed LLM-GNN integrations

that use LLMs to enhance node features for node classification or to serve directly as graph-task predictors. Chandra et al. (2020) used LLMs as text encoders and GNNs as structure encoders for fake-news detection in online communities. Chen et al. (2023b) used LLMs to annotate nodes for label-free node classification before training a GNN, and Mavromatis et al. (2023) used an LLM as a feature extractor for GNNs.

For inference, Zhu et al. (2024) proposed a textual-graph fine-tuning method that combines LLMs and GNNs through tunable side structures alongside each LLM layer, improving training efficiency and inference speed. Duan et al. (2023) introduced SimTeG, which first fine-tunes an LLM on a downstream task such as node classification and then extracts node embeddings from the final hidden states. Zhang (2023) proposed Graph-toolformer, which uses ChatGPT-driven prompt engineering to augment graph reasoning. These approaches make LLMs accessible for graph-related tasks, but they still struggle to capture complex structural detail and remain highly sensitive to prompt design.

Graph Representation as a Learned Projector Learned-projector methods provide a stronger alternative to textual descriptions by transforming graph data into embeddings or graph tokens that LLMs can consume directly. Chen et al. (2024) introduced LLAGA, a large language and graph assistant that preserves the general-purpose LLM while converting graph structures into graph tokens. (Liu et al. 2024; Qin et al. 2023) incorporated graph structure into a pre-trained LLM through tailored disentangled graph neural network layers. Tian et al. (2024) proposed graph neural prompts for encoding knowledge graphs into graph embeddings, and Huang et al. (2023b) introduced G-Prompt, a prompt-based node feature extractor for few-shot learning on text-attributed graphs that combines a learnable GNN adapter with task-specific prompts.

Another line of projector work condenses graph information into fixed-length prefixes using graph transformers attached to LLM layers (Chai et al. 2023). Yang et al. (2021) developed GraphFormers, which integrates text encoding and graph aggregation by nesting GNNs alongside transformer blocks. Peng et al. (2024) developed ChatGraph, an interactive natural-language interface for graph analysis in which graphs are represented as sequential paths and the LLM is fine-tuned on graph data.

We do not study projector adaptations that retrain the LLM because they substantially increase computational cost without providing the most relevant setting for our robustness questions. Instead, we focus on adaptation methods that use **frozen pre-trained LLMs** (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). This design reduces cost, improves compatibility with different LLM backbones, and preserves the general knowledge of the base model while adapting it to graph tasks. Throughout the paper, we use two representative projector methods: **LLAGA** (Chen et al. 2024), which uses node-level templates and a linear projector, and **GRAPHPROMPTER** (Liu et al. 2024), which uses a GNN-based projector to encode the graph.

This work. We exclude textual-description methods be-

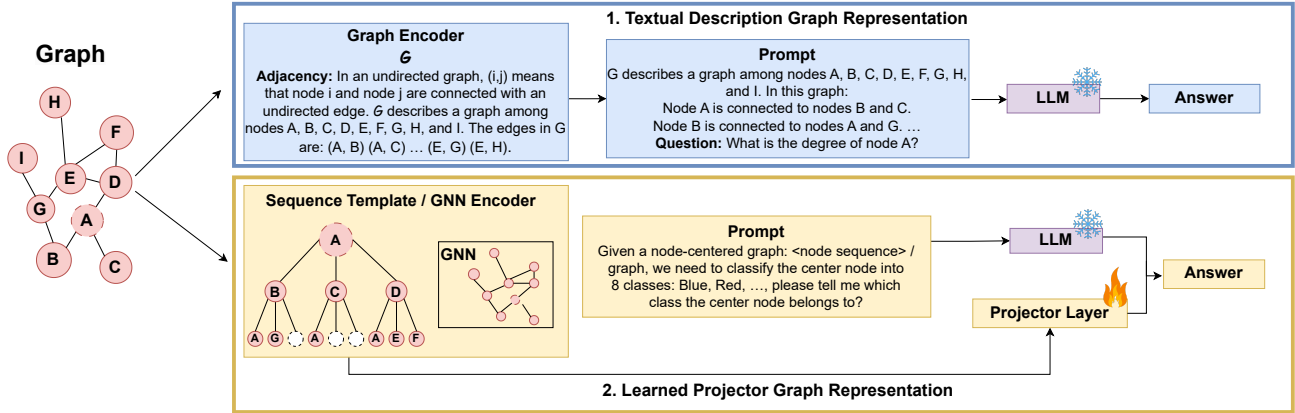


Figure 5: **Graph encoding-based adaptations of LLMs.** Textual-description methods express graphs in natural language (top). Learned-projector methods encode graphs through node templates that form sequences or through GNNs (bottom). In both cases, the LLM remains frozen and only the graph-to-token projector is trained.

Table 7: Comparison of graph encoding-based LLM adaptations.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

cause they often fail to capture the structural intricacy required by graph tasks at scale. They are best suited to simpler descriptions, such as basic degree or connectivity statements, and large graphs quickly exceed prompt-token limits when represented in detail. From a robustness perspective, textual-description methods also do not create the graph-specific attack surfaces that motivate this paper. When a graph is represented as plain text (e.g., "Node A is connected to Node B"), it is primarily exposed to conventional text-based adversarial attacks rather than the structural and feature perturbations specific to graph-based models. Projector-based graph-aware LLMs therefore provide the right setting for identifying the novel vulnerabilities studied in this work.

A.2 Attacks on GNN

GNNs are exposed to several attack families: membership inference attacks (Olatunji, Nejd, and Khosla 2021; He et al. 2021; Conti et al. 2022), which infer whether a node belonged to the training set; attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), which infer sensitive or missing node attributes from outputs or embeddings; property inference attacks (Zhang et al. 2022; Wang and Wang 2022), which extract graph-level properties; adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), which degrade performance through subtle graph or feature perturbations; and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b), which reconstruct graph structure from learned representations or model outputs.

Adversarial attacks are especially relevant here because they directly compromise model integrity. These attacks introduce small, carefully crafted *perturbations* to node features or edges with the goal of degrading GNN performance. They are commonly divided into **poisoning attacks**, which manipulate training data during the *training phase*, and **evasion attacks**, which perturb inputs at *test time*. Zügner, Akbarnejad, and Günnemann (2018) showed that small adversarial perturbations to graph structure or node features can significantly degrade GNN performance; their NETTACK method uses greedy optimization to find minimal perturbations that maximize training loss. Li et al. (2023) revisited graph adversarial attacks from a data-distribution perspective, showing how adversarial examples can alter the underlying distribution and proposing a defense to mitigate this effect. Wu et al. (2019) provided a broad analysis of graph adversarial examples and defenses, emphasizing how small perturbations create concrete vulnerabilities and how defenses can detect or counteract them. METAATTACK (Zügner and Günnemann 2019) performs both poisoning and evasion attacks by treating graph structure as an optimizable hyperparameter. It directly modifies the graph through gradient descent in a bi-level optimization problem, using meta-gradients to estimate how small structural perturbations affect the attacker’s loss after training and then selecting graph updates that degrade performance.

In this paper, we use NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019) as representative adversarial attacks be-

cause together they cover both poisoning and evasion settings. Poisoning attacks test whether training-time graph manipulation can compromise integrity, while evasion attacks test whether inference-time perturbations can degrade deployed graph-aware LLMs.

A.3 How NETTACK and METAATTACK Work

NETTACK perturbs graph structure by adding and removing edges while preserving the perturbed graph’s degree distribution. It uses greedy optimization with respect to a surrogate model to find minimal perturbations that maximize training loss. The surrogate is a linearized 2-layer graph convolutional network (GCN), which approximates how structural changes affect classification. NETTACK aims to increase confidence in incorrect classifications by maximizing the distance between logits on the perturbed and clean graphs. It identifies candidate edges around a target node, scores each candidate by its expected classification impact, and then adds or removes the highest-scoring edge. Because NETTACK is originally targeted, causing misclassification of a single node, we follow (Zügner and Günnemann 2019) and convert it into an untargeted evaluation by sequentially attacking test nodes and aggregating the resulting performance.

METAATTACK optimizes graph structure directly with gradient descent by treating the adjacency matrix as a hyperparameter. It formulates the attack as a bi-level optimization problem: the inner level minimizes training loss on the original graph, while the outer level optimizes the poisoned graph to degrade performance. A linearized two-layer GCN serves as the surrogate model for estimating how structural changes affect predictions. A score function evaluates each candidate perturbation, and meta-gradients indicate which connected-node edges should be removed or which absent edges should be added. The algorithm then greedily selects the highest-impact perturbations to maximize attack effectiveness.

For both methods, we enforce the unnoticeability constraint introduced by (Zügner, Akbarnejad, and Günnemann 2018), which limits changes to the graph degree distribution. We therefore perturb only 10% of the edges.

In poisoning experiments with LLAGA, we use the perturbed graph from NETTACK or METAATTACK to generate node-level sequences and retrain the projector that maps those sequences into token embeddings. For GRAPHPROMPTER, we re-encode the perturbed graph with a graph attention network (GAT) and retrain the projector on the perturbed graph. In evasion experiments, each graph-aware LLM is trained on clean data and receives perturbations only at inference time.

A.4 Algorithm for Node Sequence Template Injection Attacks

Algorithm 1 specifies the node sequence template injection attack for LLAGA. The model builds a *fixed-shape* computational tree for each node u by sampling k neighbors per node up to depth d (lines 1–14). If a node has fewer than k neighbors, placeholders preserve the fixed shape, and those placeholders create the exploitable attack surface. We inject malicious nodes into these positions through graph manipulation using three strategies: *Non-Adjacent Injection (NI)*

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k

Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$  neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:    $G' \leftarrow G$  ▷ Create copy of original graph
17:    $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:    $P \leftarrow \text{GetPlaceholders}(T)$ 
19:   if  $|P| > 0$  then
20:      $\mathcal{N}_2 \leftarrow$  2-hop neighborhood of  $u$ 
21:     if  $\text{AttackStrategy} = \text{NI}$  then
22:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:        $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:       for each placeholder position  $p \in P$  do
25:          $v \leftarrow$  Next node from  $V_{\text{inject}}$ 
26:         Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:       end for
28:     else if  $\text{AttackStrategy} = \text{SI}$  then
29:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:        $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:       for first placeholder position  $p \in P$  do
32:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:       break ▷ Only use first placeholder
34:     end for
35:     else if  $\text{AttackStrategy} = \text{MSI}$  then
36:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:        $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:       for  $i = 1$  to  $|P|$  do
39:          $s \leftarrow$  Get  $i$ -th node from  $S$ 
40:          $p \leftarrow$  Get  $i$ -th placeholder from  $P$ 
41:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:       end for
43:     end if
44:   end if
45:   return  $G'$ 
46: end procedure

```

inserts nodes outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)* inserts a high-degree node into one placeholder (lines 29–33); and *Multiple Supernode*

Injection (MSI) distributes distinct high-degree nodes across placeholders (lines 35–43). These uninformative nodes corrupt the target sequence representation and thereby weaken projector training and inference.

A.5 Ablation Studies

This section reports the ablation results that support the main-paper conclusions about perturbation budget and LLM backbone choice.

Ablation Results: Varying Feature Perturbation Figure 6 shows how LLAGA and GRAPHPROMPTER respond as the percentage of perturbed features changes. Increasing the perturbed-feature proportion consistently lowers accuracy for both models, confirming that feature-level perturbations are a reliable and consequential attack channel.

Ablation Results: Does the Choice of LLM Affect Attack Performance? Table 8 shows that the base LLM has minimal effect on clean accuracy and adversarial vulnerability within the LLAGA framework. Across Cora, Citeseer, and PubMed, Vicuna-7B and Llama2-7B variants achieve nearly identical clean accuracy and exhibit similar degradation under poisoning and evasion. For example, under METAATTACK evasion on Cora, performance drops to 0.44 ± 0.06 for Vicuna-7B and 0.43 ± 0.08 for Llama2-7B, a difference well within the standard deviation. The same pattern holds across datasets and attack types, reinforcing that attack effectiveness is driven primarily by graph encoding rather than by the tested base LLM.

A.6 Results on the ArXiv Dataset

We further evaluate the attack on the large-scale ArXiv dataset, which contains 169,343 nodes, 1,166,243 edges, and sparsity 0.81. This experiment tests whether node sequence template attacks remain effective on a large real-world graph. Table 9 shows a clear gap between poisoning and evasion: poisoning attacks are consistently stronger than evasion across all three injection strategies, unlike the pattern on Cora, Citeseer, and PubMed. This result indicates that ArXiv has structural characteristics that make it more vulnerable to poisoning while remaining comparatively robust to evasion. Even with very low sparsity, inserting a single supernode can substantially affect model performance.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The examples associated with Appendix A.7 show how our attacks modify real Cora node features. The “input” is the text received by the LLM after perturbation. Although a human reader sees text that is visually identical or highly similar to the original, the LLM processes different Unicode characters or a different character order, which can change its behavior. Table 10 illustrates the Homoglyph attack, and Table 11 illustrates the Reordering attack.

Feature Corrector Defense Template We design the feature corrector module to mitigate imperceptible attacks,

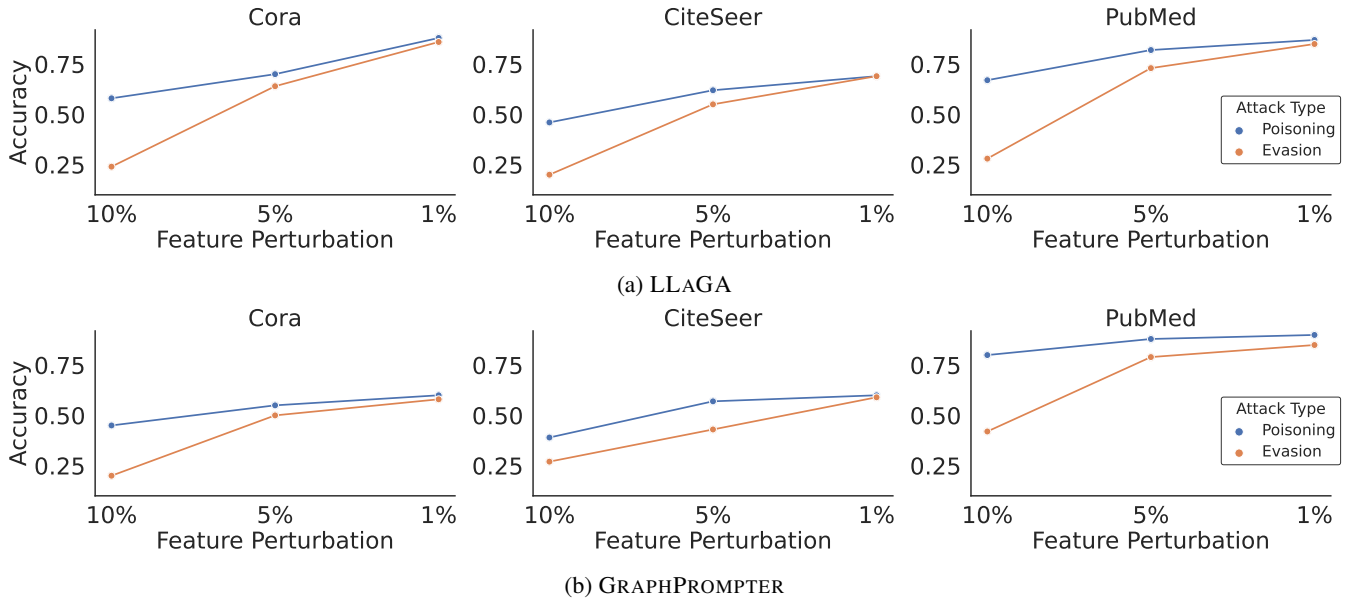


Figure 6: Accuracy of graph-aware LLMs as the percentage of perturbed features varies. Lower accuracy indicates a stronger attack.

Table 8: Effect of base LLM choice on clean and attacked LLAGA performance. The results show that changing the base model does not materially alter attack susceptibility.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

Table 9: Accuracy under node sequence template injection attacks across datasets and methods. *NI*, *SI*, and *MSI* denote non-adjacent, single-supernode, and multiple-supernode injection attacks, respectively; Clean denotes unattacked performance.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03
ArXiv	Clean	—	0.73 ± 0.02	—
	Poisoning	0.50 ± 0.03	0.43 ± 0.04	0.42 ± 0.03
	Evasion	0.71 ± 0.06	0.69 ± 0.02	0.60 ± 0.03

specifically Homoglyph substitutions and character Reorderings. The module is a prompt-based LLM defense that instructs the model to act as a linguistic validator and to detect and correct subtle perturbations that are difficult for humans to notice.

The prompt template is shown below. It states the task explicitly and enforces a clear output format, which improves correction reliability.

A.8 Detailed Discussion

Our findings establish several important design implications for graph-aware LLM robustness. As graph-structured data becomes increasingly integrated into LLM pipelines, evaluating performance alone is insufficient; the encoding mechanisms must also be tested against adversarial manipulation. Our systematic study of LLAGA and GRAPHPROMPTER reveals concrete weaknesses under poisoning and evasion attacks adapted from the GNN literature. We summarize the main implications below.

Prompt for Feature Corrector

You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.

Example 1 (Homoglyph Substitution):

Input: Title: The mgprir heuristic fr discvring prtin squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrens in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rlted squncs.

Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Example 2 (Character Reordering):

Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.

Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Example 3 (Mixed Homoglyph & Reordering):

Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In ellulr tlphne systms, n imrtnt prblm is to dynmiclly alloct th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblem is naturally frmultd s dynmic prgrmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llection plicies that ar bttr than prvius heuristic slutins.

Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.

Your Task: Text to correct: {perturbed_text_here}

Corrected Text:

Figure 7: Prompt template for the LLM-based feature corrector. The prompt explicitly defines the LLM's role, task, and expected output format.

Table 10: Imperceptible Homoglyph attack example. The LLM processes the perturbed text as distinct characters, while the rendered text remains nearly identical to a human reader.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgpprir huristic fr discvring prtln squnc ptttrns Abstract: Svrl cmputr lgrithms fr discvring pttrns in grups f prtln squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rltd squncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: Imperceptible Reordering attack example. The LLM processes a reordered character sequence, while the rendered text remains nearly identical to a human reader.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplcpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Impact of Graph Encoding on Adversarial Vulnerability

The central observation is that graph-structure integration can itself create adversarial vulnerability. The node sequence template in LLAGA creates an exploitable weakness that produces substantial performance degradation. This attack surface highlights a sharp design tradeoff: node-level templates effectively inject graph structure and can improve graph-task performance, but they also introduce attack vectors absent from traditional GNNs. By contrast, GRAPHPROMPTER, which uses a GNN-based projector, demonstrates stronger resilience, showing the robustness value of hybrid architectures that combine LLM and GNN strengths.

Sensitivity of Graph-aware LLMs to Feature Perturbations

Beyond structural encoding, the experiments show that feature perturbations are a major vulnerability for both graph-aware LLMs. Despite their different graph encoders,

LLAGA and GRAPHPROMPTER are sensitive to imperceptible node-feature changes. This result underscores a broader LLM robustness challenge: strong textual processing does not imply robustness to adversarially modified text. Subtle node-attribute changes can degrade performance without obvious human-visible artifacts, which is especially concerning for security-sensitive deployments. Text sanitization methods such as those discussed by (Boucher et al. 2022) provide partial defenses, but imperceptible attacks such as homoglyph manipulations remain difficult. Our LLM corrector directly targets this gap by prompting the model to detect and repair suspicious or malformed inputs before downstream graph processing.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs

Attack effectiveness is dataset-dependent rather than uniform. The attacks are highly

effective on Cora, Citeseer, and PubMed, while ArXiv shows a different interaction between attack strategy and graph structure. This variation demonstrates that graph-aware LLM robustness must be evaluated jointly with dataset properties, not only model architecture. As these systems are deployed across domains, vulnerability assessments must account for the structural and textual characteristics of the target graph.

Implications for Graph-aware LLM Development These vulnerabilities have direct consequences for graph-aware LLM development and deployment. As graph-aware LLMs take on tasks previously handled by GNNs, designers must treat performance and robustness as coupled objectives. The results call for stronger graph encoders, defenses that jointly address structure and text, and robustness evaluations that explicitly include feature perturbations. Understanding how GNN attacks transfer to graph-aware LLMs also opens a concrete research agenda for protecting these models against more sophisticated adversaries.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs **Defenses on adversarial attacks based on feature perturbation.** Adversarial attacks on GNNs have primarily targeted structural perturbations because feature-based perturbations often do not change the predicted class of the target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019). As a result, there are *no known defenses against feature-based attacks on GNNs* in the same sense, because those attacks were not considered a dominant threat. The graph-aware LLM setting changes this conclusion: as shown in the main paper, feature perturbations are effective and can manipulate model behavior. This creates a new defense requirement that covers both structure and text. OCR-style text sanitization can help in principle, but imperceptible attacks such as homoglyph substitutions remain difficult because OCR is imperfect and tends to misinterpret homoglyphs more often than unperturbed text (Boucher et al. 2022). This gap motivates our LLM-based self-correction mechanism for feature perturbations.

Defenses on adversarial attacks based on structural perturbation. Because structure-based attacks transfer from GNNs to graph-aware LLMs, especially under evasion, it may seem that existing GNN defenses should transfer as well. Our results show that this expectation fails. Unlike standard GNN settings, *graph-aware LLMs are also susceptible to feature perturbations, rendering these defenses ineffective* when they rely on clean textual features.

A common GNN defense makes the adjacency matrix trainable so that edge weights adapt during training to reduce adversarial impact. This strategy downweights edges between dissimilar nodes and upweights edges between feature-similar nodes. Several **preprocessing-based** defenses follow this idea, including removing low-similarity edges (Wu et al. 2019) and randomized edge dropping (Dai et al. 2018). These defenses depend on clean node features to compute meaningful similarity scores. When the features are also perturbed, as in our attacks, the defenses lose their foundation and can

even worsen the graph by preserving or creating incorrect edges. Figure 8 demonstrates this effect on a random Cora node: clean feature similarity preserves some original edges despite adding some wrong connections, whereas perturbed-feature similarity fails to preserve original connections and reconstructs an incorrect neighborhood.

Another defense family reduces reliance on graph structure. For example, (Shanthamallu, Thiagarajan, and Spanias 2021) train a node-feature-only surrogate predictor and use uncertainty matching to extract graph information from the GNN. Wu et al. (2021) introduce a co-training framework that trains feature-view and structure-view submodels independently and distills knowledge from confident unlabeled examples. These methods still assume unperturbed node features, which does not hold in our setting where both structure and features are attacked. Therefore, structural defenses developed for GNNs do not transfer directly to graph-aware LLMs; this paradigm requires defenses designed for joint structural and textual perturbations.

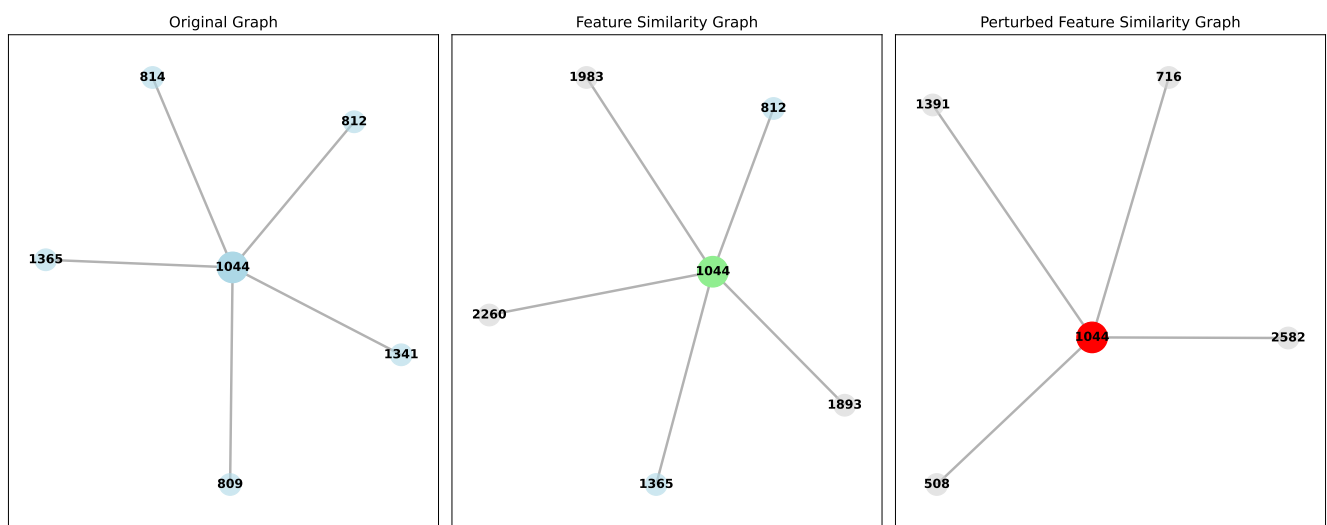


Figure 8: Edges reconstructed for random node 1044. **Original (left)** shows ground-truth edges from the original graph. **Feature Similarity (middle)** shows edges reconstructed from clean node features, with blue nodes indicating preserved original connections. **Perturbed Feature Similarity (right)** shows edges reconstructed from perturbed node features.